

Lab Test 1

[25 marks]**Coding time: 1.5 hours****General Instructions:**

1. You will perform the lab test on your personal laptop.
2. You are not allowed to communicate with anyone or access any network during the test.
3. Disable the following connections on your laptop before the test begins: Wi-Fi, Bluetooth, and any other communication devices (e.g. 3G/4G modems).
4. You may refer to any file on your laptop during the test.
5. Make sure your code can generate exactly the same output as we show in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspelling, etc. in the output.
6. Do not hardcode. We will use different test cases to test and grade your solutions.
7. Follow standard Python coding conventions (e.g., naming functions and variables).
8. Python script file that cannot be executed will NOT be marked and hence you will be awarded 0 marks. You may wish to comment out the parts in your code which cause execution errors.
9. Include your name as author in the comments of all your submitted source files.
For example, include the following block of comments at the beginning of each source file you need to submit.

```
# Name: Lum Ting Wong  
# Email ID: lum.ting.wong.2023
```

Instructions on how to submit your solutions:

1. Before the test begins, you will be instructed to download the resource files of the test from eLearn. After downloading the resource files, you should **disable the internet** on your laptop.
2. You will be given a password to unzip the downloaded .zip file. After unzipping the file, **you should rename the folder to your email id.**
For example, if your SMU email is lum.ting.wong.2023@scis.smu.edu.sg, you should rename the folder to **lum.ting.wong.2023**
You need to save your solutions to this folder for submission later. You may be penalized for not following our instructions.
3. When the test ends, you will be instructed to enable your laptop's internet and submit your solutions as a single zip file to eLearn. For example, if your folder is named lum.ting.wong.2023, you will be uploading lum.ting.wong.2023.zip

Question 1:**[8 marks]****Part (a) (Difficulty Level: *)**

A shopping centre carpark charges parking as follows:

- \$5 for the 1st hour
- \$0.10 for each subsequent minute after the 1st hour

Notes :

- no parking charges if the parking duration is within the grace period of 10 minutes (inclusive).
- parking duration beyond the grace period but within the first hour will be charged \$5.

Inside **q1a.py**, implement a function called `compute_parking()`. The function takes in 1 parameter:

- `duration` (type `int`): The parking duration in minutes. Assume this number will always be a positive integer.

The function returns the parking charges in dollars (of type `float`) given the duration in minutes.

Examples:

- `compute_parking(10)` returns `0.0`
- `compute_parking(30)` returns `5.0`
- `compute_parking(60)` returns `5.0`
- `compute_parking(70)` returns `6.0`
- `compute_parking(85)` returns `7.5`
- `compute_parking(310)` returns `30.0`

Use **q1a_test.py** to test your code. Do NOT modify `q1a_test.py`. Only **q1a.py** will be graded.

Part (b) (Difficulty Level: **)

Housing Development Board (HDB) flats are public housing in Singapore. Singaporean households living in HDB flats will receive rebates to offset **Service & Conservancy Charges (S&CC)** over 2023. The S&CC charges are paid for the purpose of essential maintenance and repairs of lifts, water pumps, lightings, lift rescue services, fumigation and refuse collection.

HDB Flat Type	No. of months of S&CC Rebate in 2023			
	January	April	July	October
1- and 2-room	0.5	1.0	1.0	1.0
3- and 4-room	0.5	1.0	0.5	0.5
5-room	0.5	0.5	0.5	0.5
Executive	-	0.5	0.5	0.5

Lab Test 1 - IS111

Example: A Singaporean household staying in a 3-room flat will receive 1.0 month of SS&C rebate in the month of April in 2023.

Note: Eligible households receive their S&CC rebate in January, April, July, and October ONLY. No rebate is provided during the other months of the year.

Inside **q1b.py**, implement a function called `compute_rebate()`. The function takes in three parameters:

- `is_singaporean_household` (type `bool`): This `bool` determines if the household is considered as a Singaporean household.
Note: To be eligible for the rebates, the household MUST be a Singaporean household.
- `flat_type` (type `str`): This string is *always* one of the following: '1', '2', '3', '4', '5', 'E'. It determines the type of the flat ('1' stands for 1-room, '2' stands for 2-room, etc., and 'E' for Executive).
- `month` (type `int`): This `int` refers to the month in 2023 and will always be a number from 1 (for January) to 12 (for December).

Note: You do not need to handle parameter values that are not listed above (e.g. '6' for flat type or 13 for month).

The function returns the number of months of S&CC rebates (type `float`) for a specific household given the 3 parameters.

Examples:

- `compute_rebate(True, "1", 7)` returns 1.0
- `compute_rebate(True, "1", 8)` returns 0.0
- `compute_rebate(False, "1", 10)` returns 0.0
- `compute_rebate(True, "2", 4)` returns 1.0
- `compute_rebate(True, "2", 7)` returns 1.0
- `compute_rebate(True, "3", 1)` returns 0.5
- `compute_rebate(True, "3", 7)` returns 0.5
- `compute_rebate(True, "4", 10)` returns 0.5
- `compute_rebate(True, "4", 12)` returns 0.0
- `compute_rebate(True, "5", 4)` returns 0.5
- `compute_rebate(True, "5", 11)` returns 0.0
- `compute_rebate(True, "E", 1)` returns 0.0

Use **q1b_test.py** to test your code. Do NOT modify `q1b_test.py`. Only **q1b.py** will be graded.

Question 2:**[8 marks]****Part (a) (Difficulty Level: *)**

In **q2a.py**, implement a function called `get_digits_of_even_index()`. The function takes in a string called `my_str` as its parameter. The function extracts all the digits located at **even** positions (or indexes) in the string.

The function returns the extracted digits as a string.

If `my_str` is empty (or does not contain any digit), the function returns empty string (`''`).

Examples:

- `get_digits_of_even_index('123456')` returns `'135'`, because digits in `'123456'` located at even indexes are `'1'`, `'3'`, and `'5'`.
- `get_digits_of_even_index('SMU-1-2-3-4')` returns `'1234'`, because digits in `'SMU-1-2-3-4'` located at even indexes are `'1'`, `'2'`, `'3'`, and `'4'`.
- `get_digits_of_even_index('32!*-4*5(0)36')` returns `'36'`, because digits in `'32!*-4*5(0)36'` located at even indexes are `'3'` and `'6'`.
- `get_digits_of_even_index('2023SMU-SCIS1')` returns `'221'`, because digits in `'2023SMU-SCIS1'` located at even indexes are `'2'`, `'2'`, and `'1'`.
- `get_digits_of_even_index('SMU-SCIS')` returns `''`, because there is no digit in `'SMU-SCIS'`.
- `get_digits_of_even_index('SCIS-1')` returns `''`, because although there is the digit `'1'` in `'SCIS-1'`, it is not located at an even index.
- `get_digits_of_even_index('')` returns `''` because the string is empty.

Use **q2a_test.py** to test your code. Do NOT modify `q2a_test.py`. Only **q2a.py** will be graded.

Part (b) (Difficulty Level: **)

In **q2b.py**, implement a function called `count_non_digits()`. The function takes a single parameter called `str_list`, which is a list of strings.

The function counts the number of non-digit characters in each string of `str_list`.

The function returns a list containing the count of non-digit characters for each string in `str_list`.

If `str_list` is empty the function returns an empty list.

Lab Test 1 - IS111

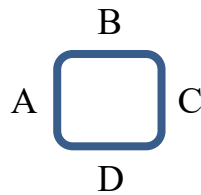
Examples:

- `count_non_digits(['123456', '13579', '24680', '888'])` returns `[0, 0, 0, 0]`, because:
 - The 4 strings of the list have 0 non-digit character.
- `count_non_digits(['abc', 'Hello Python!', ''])` returns `[3, 13, 0]`, because:
 - The first string 'abc' has 3 non-digit characters.
 - The second string 'Hello Python!' has 13 non-digit characters.
 - The third string '' has 0 non-digit character.
- `count_non_digits(['1-2-3-4-5-6', '32!*-4*5*6-', '2023-24-T1-SMU-SCIS'])` returns `[5, 6, 12]`, because:
 - The first string '1-2-3-4-5-6' has 5 non-digit characters.
 - The second string '32!*-4*5*6-' has 6 non-digit characters.
 - The third string '2023-24-T1-SMU-SCIS' has 12 non-digit character.
- `count_non_digits(['', '', ''])` returns `[0, 0, 0]`, because there are 0 non-digit characters in each string of the given list.
- `count_non_digits([])` returns `[]` because `str_list` is empty.

Use **q2b_test.py** to test your code. Do NOT modify `q2b_test.py`. Only **q2b.py** will be graded.

Question 3:**[5 marks]**

SMU is organising a workshop open to all its students and to the students doing an exchange semester at SMU. The person in charge of this workshop would like to form groups of four students per table as shown below:



To optimize the outcome of the workshop, the organiser would like the grouping of students to follow certain rules, as explained in the following 3 questions.

Part (a) (Difficulty Level: ** / *)**

Inside `q3a.py`, implement a function called `is_partially_compliant()`:

- The function takes in a single parameter called `table`, which is a list of 4 tuples representing the 4 students seated at the table. The first tuple represents the student seated on the left side of the table; and the following tuples represent the students seated at the seats following the clockwise direction. For example, for the diagram above, student A would be the first tuple in the list, followed by student B, and C, and student D would be the last tuple in the list.
- Each tuple represents the characteristics of the student as follows:
 - name of the student (type: `str`),
 - year when the student did his/her last internship (type: `int`),
 - school which the student belongs to (type: `str`),
 - the languages that the student speaks (type: `list`),
 - the skills that the student mainly practiced during his/her last internship (type: `list`)

For example, the tuple `('A1', 2022, 'bus', ['english', 'mandarin'], ['planning', 'decision making', 'data analysis', 'oral communication'])` represents a student called A1, who did his/her last internship in 2022, who is a student from the business school, who speaks English and Mandarin and who practiced during his/her last internship the following skills: 'planning', 'decision making', 'data analysis', and 'oral communication'.

Note:

- The number of languages and skills may vary from one student to another. You can assume that those 2 lists are not empty and are not limited.
- The schools of SMU are noted as follows: 'scis', 'bus', 'acc', 'sosc', 'law', 'eco' for school of computing and information systems, school of business, school of accountancy, school of social sciences, school of law, and school of economics, respectively.
- The school of an exchange student will be the school where this student makes the exchange at SMU (meaning one of the 6 SMU schools listed above).
- The order of the languages and the skills in the lists are not important.
- Assume all the languages spoken by each student as well as the skills practiced during their last internship are captured in lowercase letters.
- Assume that all the students seated at the table have completed an internship.
- The function returns `True` if the table is deemed partially compliant and `False` otherwise. A table is deemed partially compliant if **both** the following conditions are satisfied:

Lab Test 1 - IS111

- Among the 4 students sitting at the table, there are **at most 2 students from the same school**. In other words, for each of the schools listed, there should be no more than 2 students from that school. Indeed, the organizer wants to ensure that the background of the students at the same table is diversified. So, no table should get 3 or 4 students coming from the same school.
- The 4 students have done their **last internship either** in the same year or within a maximum of 2 consecutive years of each other. Indeed, the organiser wants to ensure that the students had done their last internship in a similar timeframe.

Examples:

- `table_1 = [('A1', 2021, 'scis', ['english', 'german'], ['data analysis', 'python']), ('B1', 2022, 'bus', ['english', 'french'], ['teamwork', 'oral communication']), ('C1', 2022, 'scis', ['chinese', 'english'], ['data analysis']), ('D1', 2021, 'bus', ['tamil','english', 'italian'], ['teamwork', 'time management'])]`

`is_partially_compliant(table_1)` returns True because in this table:

- The 4 students have done their last internship in 2021 or 2022 (at most 2 consecutive years) and
- At most 2 students are from the same school ('scis' and 'bus').

- `table_2 = [('A2', 2020, 'scis', ['english', 'german'], ['data analysis', 'python']), ('B2', 2022, 'bus', ['english', 'french'], ['teamwork', 'oral communication']), ('C2', 2022, 'scis', ['chinese', 'english'], ['data analysis']), ('D2', 2020, 'bus', ['tamil','english', 'italian'], ['teamwork', 'time management'])]`

`is_partially_compliant(table_2)` returns False because in this table:

- The 4 students have done their last internship in 2020 or 2022 (at most 2 different years) but 2020 and 2022 are NOT consecutive years.

- `table_3 = [('A3', 2020, 'scis', ['english', 'german'], ['data analysis', 'python']), ('B3', 2021, 'bus', ['english', 'french'], ['teamwork', 'oral communication']), ('C3', 2021, 'scis', ['chinese', 'english'], ['data analysis']), ('D3', 2022, 'bus', ['tamil','english', 'italian'], ['teamwork', 'time management'])]`

`is_partially_compliant(table_3)` returns False because in this table:

- The 4 students have done their last internship in 2020, 2021 and 2022 (3 different years).

- `table_4 = [('A4', 2023, 'scis', ['english', 'german'], ['data analysis', 'python']), ('B4', 2022, 'scis', ['english', 'french'], ['teamwork', 'oral communication']), ('C4', 2022, 'scis', ['chinese', 'english'], ['data analysis']), ('D4', 2022, 'bus', ['tamil','english', 'italian'], ['teamwork', 'time management'])]`

`is_partially_compliant(table_4)` returns False because in this table:

- There are more than 2 students from the same school (3 students are from 'scis').

Use `q3a_test.py` to test your code. Do NOT modify `q3a_test.py`. Only `q3a.py` will be graded.

Lab Test 1 - IS111

Part (b) (Difficulty Level: ** / *)**

Inside `q3b.py`, implement a function called `is_fully_compliant()`:

- The function takes in a single parameter called `table`, (as described in Part (a)).
- The function returns `True` if the function satisfies ALL the following criteria and `False` otherwise:
 - The table must be partially compliant (as described in Part a)
 - If 2 students are sitting next to each other, they must:
 - Speak at least one common language (to be able to communicate)
 - Have practiced at most 2 same skills during their last internship (to ensure a diversity of competencies at the same table).

Examples:

- `table_10 = [('A10', 2021, 'scis', ['english', 'german'], ['data analysis', 'teamwork', 'problem solving', 'python']), ('B10', 2022, 'bus', ['english', 'french'], ['teamwork', 'problem solving', 'oral communication']), ('C10', 2022, 'scis', ['chinese', 'english'], ['data analysis', 'problem solving', 'python', 'java']), ('D10', 2021, 'bus', ['tamil', 'english', 'italian'], ['teamwork', 'time management', 'data analysis'])]`

`is_fully_compliant(table_10)` returns `True` because:

- all the criteria are satisfied

- `table_11 = [('A11', 2021, 'scis', ['english', 'german'], ['data analysis', 'teamwork', 'problem solving', 'python']), ('B11', 2022, 'bus', ['english', 'french'], ['teamwork', 'problem solving', 'oral communication']), ('C11', 2023, 'scis', ['chinese', 'english'], ['data analysis', 'problem solving', 'python', 'java']), ('D11', 2021, 'bus', ['tamil', 'english', 'italian'], ['teamwork', 'time management', 'data analysis'])]`

`is_fully_compliant(table_11)` returns `False` because:

- `table_11` is not partially compliant

- `table_12 = [('A12', 2021, 'scis', ['english', 'german'], ['data analysis', 'teamwork', 'problem solving', 'python']), ('B12', 2022, 'bus', ['english', 'french'], ['teamwork', 'problem solving', 'oral communication']), ('C12', 2022, 'scis', ['chinese', 'english'], ['data analysis', 'problem solving', 'python', 'java']), ('D12', 2021, 'bus', ['tamil', 'english', 'italian'], ['teamwork', 'time management', 'data analysis', 'python'])]`

`is_fully_compliant(table_12)` returns `False` because:

- 'A12' and 'D12' have practiced the same 3 skills during their last internship ('data analysis', 'teamwork' and 'python').

Lab Test 1 - IS111

- `table_13 = [('A13', 2021, 'scis', ['english', 'german'], ['data analysis', 'python']), ('B13', 2022, 'bus', ['chinese', 'french'], ['teamwork', 'problem solving', 'oral communication']), ('C13', 2022, 'scis', ['chinese', 'english'], ['data analysis', 'problem solving', 'python', 'java']), ('D13', 2021, 'bus', ['tamil', 'english', 'italian'], ['teamwork', 'time management', 'data analysis'])]`

`is_fully_compliant(table_13)` returns `False` because:

- 'A13' and 'B13' don't speak a common language

Note:

You can still do `q3b.py` even if you didn't manage to code properly the function

`is_partially_compliant()` in `q3a.py`. When we mark `q3b.py`, we will use our own `q3a.py` file.

Use `q3b_test.py` to test your code. Do NOT modify `q3b_test.py`. Only `q3b.py` will be graded.

Part (c) (Difficulty Level: ** / *)**

Inside `q3c.py`, implement a function called `make_tables_fully_compliant()`.

- The function takes in two parameters called `t1` and `t2`: two tables as described earlier for the parameter table in Parts a and b.
- Assume that at least one table does not satisfy the requirements mentioned in Part b.

The function `make_tables_fully_compliant()` checks if it is possible to swap ONE student in `t1` with ONE student in `t2` such that after the swapping, **both** tables satisfy the requirements mentioned in Part b (meaning that the function `is_fully_compliant()` returns `True` for both tables after the swap).

Note that it is not allowed to make any other change with the two tables besides swapping those two students.

- If such a swap can be found, then the function returns a tuple containing 3 elements:
 - A tuple containing the names of the 2 students to be swapped, where the first name is the name of the student from `t1` and the second name is the name of the student from `t2`.
If there are multiple ways to do the swapping, the function only needs to return one of such swapping.
 - `new_t1`: a new table `t1` including the student's swap (coming from `t2`) making the `new_t1` fully compliant (the students of `t1` not involved in the swap remain unchanged in content and position in the list).
 - `new_t2`: a new table `t2` including the student's swap (coming from `t1`) making the `new_t2` fully compliant (the students of `t2` not involved in the swap remain unchanged in content and position in the list).

Note: The function should not return `t1` nor `t2` but the `new_t1` and `new_t2` which contain the students' swap (making the new tables fully compliant).

Lab Test 1 - IS111

- The function returns the string "not possible" if such a swapping cannot be found.

A function `test_part_c()` has been given to you in `q3c_test.py` to test whether 2 tables can become fully compliant after swapping.

Examples:

- Running `q3c_test.py` will call the given function `test_part_c()` 3 times with the following arguments:
 - o `test_part_c(table_12, table_13)`
 - o `test_part_c(table_10, table_12)`
 - o `test_part_c(table_11, table_12)`
 which produces the following output:

```
First table fully compliant: False
Second table fully compliant: False
*****
* Swap done: ('A12', 'A13') *
*****
First table fully compliant: True
Second table fully compliant: True
```

```
First table fully compliant: True
Second table fully compliant: False
*****
* Swap done: ('C10', 'A12') *
*****
First table fully compliant: True
Second table fully compliant: True
```

```
First table fully compliant: False
Second table fully compliant: False
*****
* Swap done: not possible *
*****
```

Note:

You can still do `q3c.py` even if you didn't manage to code properly the function `is_fully_compliant()` in `q3b.py` or `is_partially_compliant()` in `q3a.py`. When we mark `q3c.py`, we will use our own `q3a.py` and `q3b.py` files.

Use `q3c_test.py` to test your code. Do NOT modify `q3c_test.py`. Only `q3c.py` will be graded.

Question 4:**[4 marks]****(Difficulty Level: ***)**

A food delivery company wants to use autonomous robots for the delivery of food items. For the trial run, there is only one pick-up point at the collection centre and there is only one robot.

In `q4.py`, define a function called `schedule_delivery()`. The function takes in a parameter called `items`, which is a list of tuples. Each tuple contains the following 4 elements:

- `item_name` (type: `str`): The name of the item to be delivered.
- `destination` (type: `str`): The name of the item's destination.
- `latest_delivery_time` (type: `str`): The item's latest delivery time at destination, as requested by the customer, in the format HH:MM.
- `travel_time` (type: `int`): The time taken (in minutes) to travel from the collection centre to the destination.

The function returns the schedule as a list of tuples, with each tuple consisting of 4 elements:

- `item_name` (type: `str`): The name of the item to be delivered.
- `destination` (type: `str`): The name of the item's destination.
- `latest_delivery_time` (type: `str`): The item's latest delivery time at destination, as requested by the customer, in the format HH:MM.
- `actual_delivery_time` (type: `str`): The item's actual delivery time, in the format HH:MM.

The function must fulfil the following constraints:

1. Assume that the food delivery company only accepts orders where the `latest_delivery_time` ranges from 07:00 to 21:00.
2. The robot delivers one item at a time to each destination and promptly returns to the collection center after each delivery. You can assume the time taken to return to the collection centre from a particular destination is the same as the specified item's `travel_time`.
3. To ensure food safety, the item should be delivered at its destination no earlier than 30 minutes before `latest_delivery_time`, that is, the item should not be dropped off too early so that it does not end up waiting for its recipient for too long.
4. The first delivery will always arrive at exactly 30 minutes before the stipulated `latest_delivery_time`.
5. Each item's `travel_time` will never exceed 30 minutes.
6. The destinations are unique, that is, there will not be any duplicates in the specified destinations.
7. If the item's `actual_delivery_time` is later than its `latest_delivery_time` requested by the customer, there should be an asterisk `'*'` appended after `actual_delivery_time` for easy identification.

Note:

There are various techniques for task scheduling. For this question, you are **required** to start with **sorting** the list of tuples in an **increasing** order of their **latest delivery times**. A function `sort_by_latest_delivery_time()` has been provided for you in the file `q4.py`. This function sorts a list of items in an increasing order of their `latest_delivery_time`. Please refer to the docstrings of this function for more information. You **MUST** use this function in your solution and you are **NOT** allowed to modify the code in this function.

Lab Test 1 - IS111

Scenario 1

If the list of items is defined as:

```
items = [
    ('Item 1', 'Destination A', '12:30', 15),
    ('Item 2', 'Destination B', '13:30', 10),
    ('Item 3', 'Destination C', '13:15', 15),
    ('Item 4', 'Destination D', '13:45', 20)
]
```

Then the schedule for the robot would be:

```
[('Item 1', 'Destination A', '12:30', '12:00'),
 ('Item 3', 'Destination C', '13:15', '12:45'),
 ('Item 2', 'Destination B', '13:30', '13:10'),
 ('Item 4', 'Destination D', '13:45', '13:40')]
```

- In the scenario above, the robot delivered the items in an increasing order of their `latest_delivery_time`.
- Item 1 was delivered with an `actual_delivery_time` of '12:00' according to constraints 3 and 4.
- Item 3 was delivered with an `actual_delivery_time` of '12:45' as, according to constraint 3, the actual delivery time of Item 3 should not be earlier than 12:45 (which is 30 mins before its latest delivery time of 13:15).
- Do not assume that there are always 4 items to be delivered.

Scenario 2

If the list of items is defined as:

```
items = [
    ('Item 1', 'Destination A', '12:30', 15),
    ('Item 2', 'Destination B', '13:45', 15),
    ('Item 3', 'Destination C', '13:15', 30),
    ('Item 4', 'Destination D', '13:45', 30)
]
```

Then the schedule for the robot would be:

```
[('Item 1', 'Destination A', '12:30', '12:00'),
 ('Item 3', 'Destination C', '13:15', '12:45'),
 ('Item 2', 'Destination B', '13:45', '13:30'),
 ('Item 4', 'Destination D', '13:45', '14:15*')]
```

- In the scenario above, both Item 2 and Item 4 have the same `latest_delivery_time`. The robot will deliver based on the order specified in the initial list, therefore it will deliver Item 2 followed by Item 4.
- Item 4 was delivered with an `actual_delivery_time` of '14:15'. Since the `actual_delivery_time` is later than its `latest_delivery_time`, there should be an asterisk '*' appended after `actual_delivery_time`, as shown above.

Use `q4_test.py` to test your code. Do NOT modify `q4_test.py`. Only `q4.py` will be graded.

*** END OF PAPER ***